



@qa_insights

JAVA

COMPLETE NOTES

(Java)

part 1



BEGINNER TO ADVANCED

SWIPE TO NEXT →



FOLLOW



SHARE



COMMENT



LIKE

Introduction to Java

① What is Java?

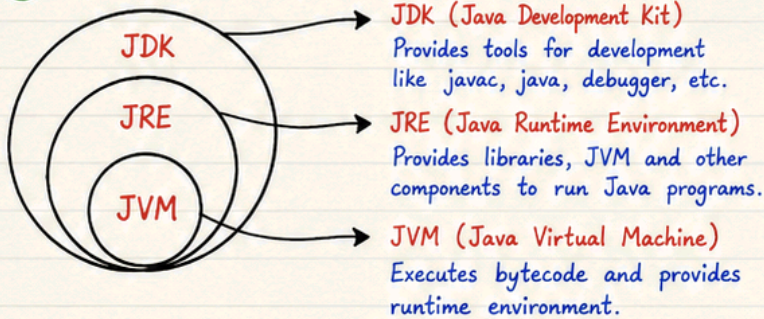
Java is a high-level, class-based, object-oriented programming language designed by Sun Microsystems (now owned by Oracle). It is platform independent, secure, robust and widely used for building enterprise applications, Android apps, web apps and more.



② Features of Java

- ✓ Simple
- ✓ Object-Oriented (OOP)
- ✓ Platform Independent
- ✓ Secure
- ✓ Robust
- ✓ Multithreaded
- ✓ Architecture Neutral
- ✓ Portable
- ✓ High Performance
- ✓ Distributed
- ✓ Dynamic
- ✓ Automatic Memory Management (GC)

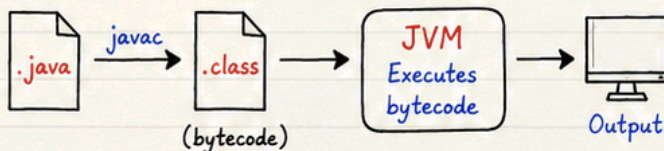
③ JDK vs JRE vs JVM



Component	Contains
JDK	JRE + JVM + Development Tools (javac, jar, javadoc, etc.)
JRE	JVM + Core Libraries + Other Runtime Components
JVM	Class Loader + Bytecode Verifier + Execution Engine + Runtime Data Areas

④ How Java Works?

Java code (.java) is compiled into bytecode (.class) using the Java Compiler (javac). This bytecode is executed by JVM on any platform.



⑤ Compile & Execution Flow

1. Write Java code (.java)
2. Compile using **javac** (Compiler)
3. Generates **bytecode** (.class file)
4. **JVM** loads and verifies bytecode
5. **JVM** executes bytecode
6. Output is displayed

⑥ First Java Program ★

```
// Hello World Program
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

- class keyword → defines a class
- main() → entry point of program
- System.out.println() → prints output
- Every statement ends with ;
- File name must match class name (HelloWorld.java)

✓ Remember

- Java is "Write Once, Run Anywhere" (WORA).
- JVM makes Java platform independent.
- Bytecode is the heart of Java.

★ Interview Fact

Java was designed by James Gosling in 1995 and first released in 1996. The file extension for Java source code is **.java** and for bytecode is **.class**

⚠ Common Mistake

- Filename and class name mismatch.
- Missing main() method.
- Not understanding compilation vs execution.

JDK, JRE & JVM

① JDK (Java Development Kit)

- Full package for Java development.
- Includes JRE + development tools.
- Used to write, compile and run Java programs.

Includes:

- JRE
- Compiler (javac)
- Tools (javadoc, jar, jdb, etc.)

② JRE (Java Runtime Environment)

- Provides libraries, JVM and other components to run Java applications.
- Does not include development tools.

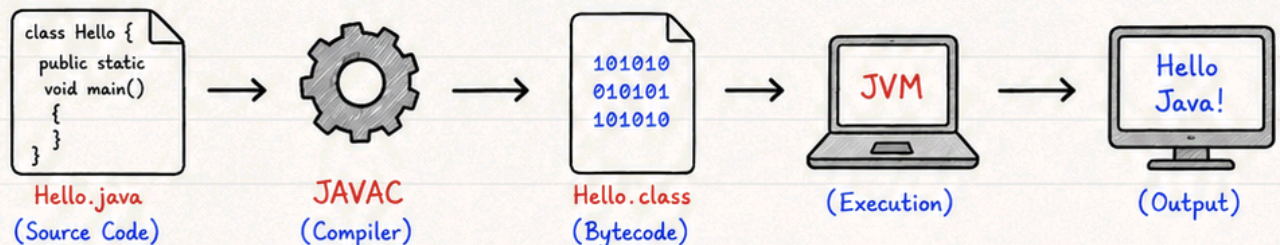
Includes:

- JVM
- Core Libraries (rt.jar, etc.)

③ JVM (Java Virtual Machine)

- Heart of Java.
- Executes Java bytecode.
- Platform independent - Write Once, Run Anywhere.
- Manages memory, threads, security and more.

④ Compilation Process



⑤ Bytecode

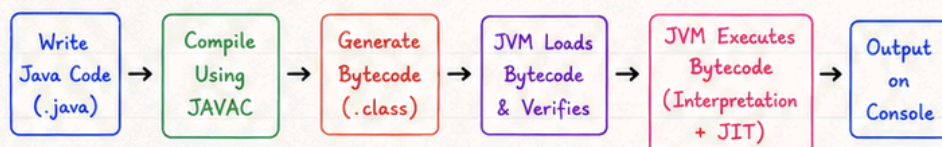
- Output of Java compiler (javac).
- Platform independent.
- Executed by JVM, not by OS directly.
- File extension: .class



Important Note

JVM converts bytecode into machine code at runtime using **JIT (Just-In-Time)** compiler for better performance.

⑥ Execution Flow Diagram



Java is **Platform Independent** because JVM is available on all major OS.



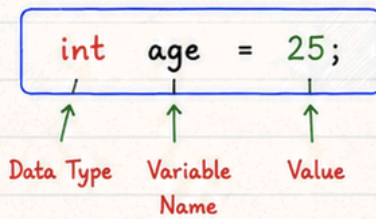
Key Takeaway

- ✓ Develop in **JDK**
- ✓ Run in **JRE**
- ✓ Execution happens inside **JVM**
- ✓ Write Once, Run Anywhere

Variables & Data Types

① What is a Variable?

A variable is a named location in memory used to store data that can be changed during program execution.



② Data Types

a) Primitive Types

Store single values.

Type	Size	Default Value
byte	1 byte	0
short	2 bytes	0
int	4 bytes	0
long	8 bytes	0L
float	4 bytes	0.0f
double	8 bytes	0.0d
char	2 bytes	'\u0000'
boolean	1 bit	false

b) Non-Primitive Types

Store reference to objects.

- String
- Array
- Class
- Interface
- Object
- Collection (List, Set, Map)
- etc.

③ Literals

- Integer Literal : 25, -10, 0
- Floating Literal : 3.14, -0.5f
- Character Literal : 'A', 'z', '\n'
- String Literal : "Hello Java"
- Boolean Literal : true, false

④ Naming Rules for Variables

- ☒ Must start with a letter, '\$' or '_'
- ☒ Can contain letters, digits, '_' and '\$'
- ☒ Case sensitive: age and Age are different
- ☒ Cannot be a Java keyword (int, class, etc.)
- ☒ Examples of invalid names: 1age, int, @age

⑤ Memory Basics

	Memory	
1001 →	25	→ <code>int age = 25;</code>
1002 →	3.14	→ <code>double pi = 3.14;</code>
1003 →	'A'	→ <code>char grade = 'A';</code>
1004 →	true	→ <code>boolean isActive = true;</code>
	...	



Note:

Each variable occupies memory based on its data type. Primitive types store actual values. Non-primitive types store reference (address) to objects in memory.

★ Interview Tip

```
int a = 10;
int b = a; // b gets a copy of value
a = 20;    // b remains unchanged
System.out.println(b); // Output: 10
```

★ Best Practices

- ☒ Use meaningful variable names.
- ☒ Declare variables close to where they are used.
- ☒ Prefer final for constants.
- ☒ Follow camelCase naming convention.

Operators & Type Casting

① Arithmetic Operators

Used to perform basic mathematical operations.

Operator	Description	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus (Remainder)	$a \% b$
++	Increment	$a++$
--	Decrement	$a--$

② Relational Operators

Used to compare two values. Result is boolean (true/false).

Operator	Description	Example
==	Equal to	$a == b$
!=	Not equal to	$a != b$
>	Greater than	$a > b$
<	Less than	$a < b$
>=	Greater than or equal	$a >= b$
<=	Less than or equal	$a <= b$

③ Logical Operators

Used to combine multiple conditions.

Operator	Description	Example
&&	Logical AND	$(a > 0) \&\& (b > 0)$
	Logical OR	$(a > 0) (b > 0)$
!	Logical NOT	$!(a > 0)$

④ Assignment Operators

Used to assign values to variables.

Operator	Example	Same As
=	$a = b$	$a = b$
+=	$a += b$	$a = a + b$
-=	$a -= b$	$a = a - b$
*=	$a *= b$	$a = a * b$
/=	$a /= b$	$a = a / b$
%=	$a \% = b$	$a = a \% b$

⑤ Unary Operators

Operate on a single operand.

Operator	Description	Example
+	Unary Plus	$+a$
-	Unary Minus	$-a$
++	Pre-Increment	$++a$
--	Pre-Decrement	$--a$
!	Logical Complement	$!true \rightarrow false$
~	Bitwise Complement	$\sim a$

💡 Example Code

```
int a = 10, b = 3;
System.out.println("a + b = " + (a + b)); // 13
System.out.println("a - b = " + (a - b)); // 7
System.out.println("a * b = " + (a * b)); // 30
System.out.println("a / b = " + (a / b)); // 3
System.out.println("a % b = " + (a % b)); // 1
System.out.println("a > b : " + (a > b)); // true
```

★ Interview Tip

Operators are the building blocks of expressions and conditions in Java. Understand precedence and type casting to avoid bugs.

⑥ Type Casting

Converting one data type into another.

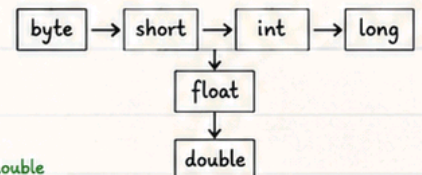
- **Implicit (Widening)** - done automatically.
- **Explicit (Narrowing)** - done manually.

1) Implicit (Widening)

Smaller $\rightarrow$ Larger type

Example:

```
int a = 10;
double b = a; // int to double
```

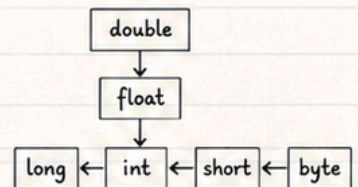


2) Explicit (Narrowing)

Larger $\rightarrow$ Smaller type (Manual)

Example:

```
double x = 25.75;
int y = (int) x; // double to int
```



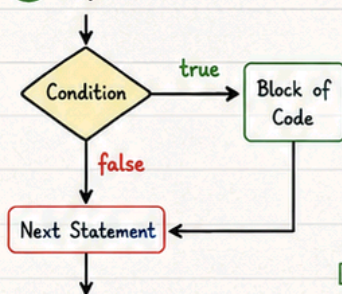
Common Mistake

Always be careful while type casting. Narrowing conversions may lead to data loss.

Control Statements

Control statements change the flow of execution based on conditions or perform repeated execution of a block of code.

① if

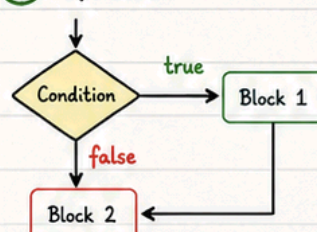


```

int age = 18;
if (age >= 18) {
    System.out.println(
        "Eligible to Vote");
}
  
```

✓ Executes block if condition is true.

② if-else

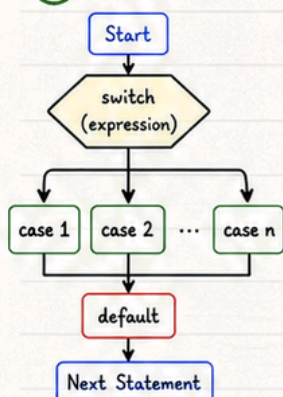


```

int n = -5;
if (n >= 0) {
    System.out.println("Positive");
} else {
    System.out.println("Negative");
}
  
```

✓ Executes one block from two based on condition.

③ switch



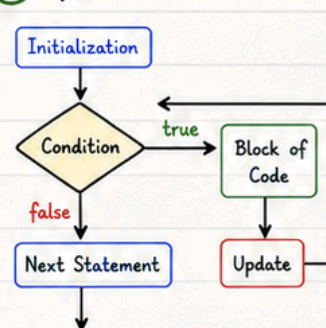
```

int day = 2;
switch (day) {
    case 1 -> System.out.println("Mon");
    case 2 -> System.out.println("Tue");
    case 3 -> System.out.println("Wed");
    default -> System.out.println("Invalid");
}
  
```

✓ Used when we have multiple possible values to match.

Note:
In Java 17+, switch supports arrow (->) syntax and works with String, enum also.

④ for

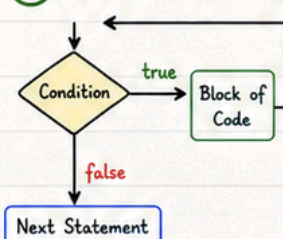


```

for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
  
```

✓ Best when number of iterations is known.

⑤ while

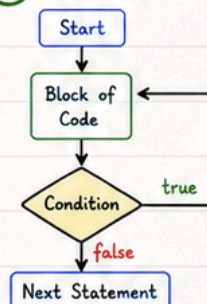


```

int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
  
```

✓ Executes block while condition is true.

⑥ do-while



```

int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 5);
  
```

✓ Executes block at least once, then checks condition.

⑦ break & continue

- **break**
Terminates the loop or switch immediately.

```

for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.print(i + " ");
}
  
```

★ Output: 1 2

- **continue**
Skips current iteration and moves to next iteration.

```

for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.print(i + " ");
}
  
```

★ Output: 1 2 4 5



Interview Tip: Choose the right control statement to improve code readability and performance.

✓ **Best Practice:** Avoid deep nesting. Use early return, break, or continue to simplify logic.

Methods in Java



A method is a block of code that performs a specific task.

① Method Syntax

```
accessModifier returnType methodName(parameterList) {
    // method body
    return value;
}
```

public, private
protected, etc.

int, void,
String, etc.

Name of
the method

Input
values

② Parameters

- Parameters are used to pass data into a method.
- They act as placeholders.

```
public int add(int a, int b) {
    return a + b;
}
```

a and b are parameters

③ Return Types

- Method can return a value to the caller using **return** statement.

Return Type	Meaning
void	No value is returned
int, double, etc.	Returns numeric value
String	Returns text value
boolean	Returns true or false

Example

```
public int square(int n) {
    return n * n;
}
```

④ Method Overloading

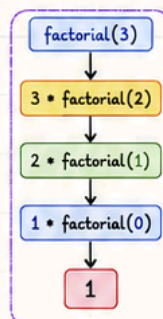
- Same method name with different parameters.
- Improves code readability.

```
int add(int a, int b) {
    return a + b;
}
int add(int a, int b, int c) {
    return a + b + c;
}
```

⑤ Recursion

- A method calling itself.
- Used to solve complex problems in a simple way.

```
int factorial(int n) {
    if (n == 0) return 1; // Base Case
    return n * factorial(n - 1); // Recursive Call
}
```



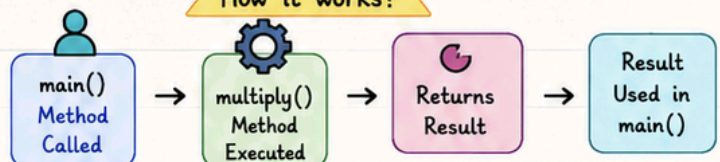
⑥ Variable Scope

Type	Scope	Lifetime	Example
Local Variable	Inside method or block	Created when method/block is called	int x = 10;
Instance Variable (Non-static)	Inside class, outside methods	Exists as long as object exists	int age;
Static Variable (Class Variable)	Whole class	Exists till program ends	static int count;

⑦ Method Example (Complete Flow)

```
public class Demo {
    public static int multiply(int a, int b) {
        return a * b;
    }
    public static void main(String[] args) {
        int result = multiply(5, 4); // method call
        System.out.println("Result: " + result);
    }
}
```

How it works?



Interview Tip

Methods help in code reusability, maintainability and modular programming.



★ Best Practices

- Use meaningful method names.
- Keep methods small and focused.

- Avoid duplicate code.
- Follow single responsibility principle.



Remember:

A good method makes your code clean and easy to understand.

Arrays in Java

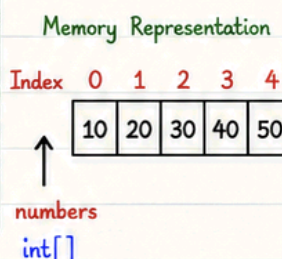
An array is a collection of similar data types stored in contiguous memory locations.

① 1D Arrays

- Stores elements of the same type in a single row.
- Index starts from 0.

Example:

```
int[] numbers = new int[5];
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
numbers[3] = 40;
numbers[4] = 50;
```

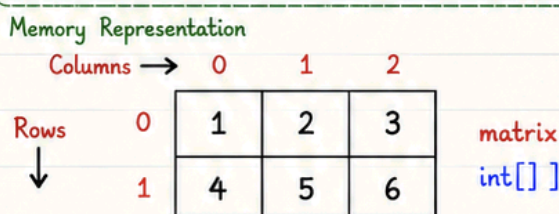


② 2D Arrays

- Array of arrays (rows and columns).
- Useful for tables, matrices, grids.

Example:

```
int[][] matrix = new int[2][3];
matrix[0][0] = 1; matrix[0][1] = 2; matrix[0][2] = 3;
matrix[1][0] = 4; matrix[1][1] = 5; matrix[1][2] = 6;
```



③ Array Operations

Operation	Description	Example
Declare	Create an array	<code>int[] arr = new int[5];</code>
Access	Access element using index	<code>int val = arr[2];</code>
Update	Update element at index	<code>arr[2] = 100;</code>
Length	Get total number of elements	<code>int len = arr.length;</code>
Traversal	Loop through all elements	<code>for (int i = 0; i < arr.length; i++) { ... }</code>

④ for-each Loop

- Simplified way to traverse arrays.
- Read-only access (cannot modify index).

Example:

```
int[] arr = {10, 20, 30, 40, 50};
for (int num : arr) {
    System.out.println(num);
}
```

★ Output

10
20
30
40
50

⑤ Arrays Utility Class

- `java.util.Arrays` provides useful methods to work with arrays.

Method	Description
<code>Arrays.toString(arr)</code>	Returns string representation
<code>Arrays.sort(arr)</code>	Sorts array in ascending order
<code>Arrays.fill(arr, val)</code>	Fills array with given value
<code>Arrays.copyOf(arr, newLen)</code>	Copies array to new length
<code>Arrays.equals(arr1, arr2)</code>	Compares two arrays



Interview Tip

- Arrays are fixed in size.
- `IndexOutOfBoundsException` occurs for invalid index.
- Use for-each loop for simple traversal.
- Arrays are objects in Java.



Best Practices

- Use meaningful names.
- Always check index before accessing.
- Prefer Arrays utility methods for operations.
- Avoid storing large data in stack, use arrays in heap memory.

Strings in Java

① String

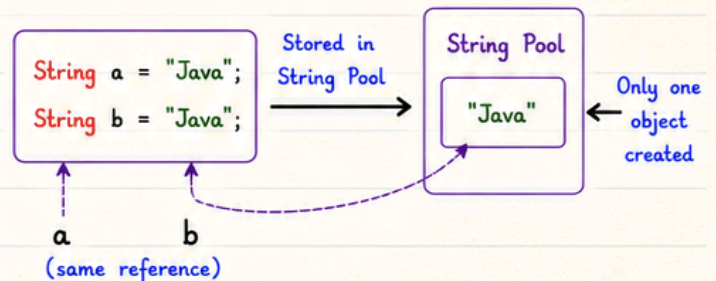
- A String is a sequence of characters.
- In Java, String is an object of class `java.lang.String`.
- Strings are widely used to store and manipulate text.

Example:

```
String name = "Java";
String msg = new String("Hello");
System.out.println(name);    // Java
System.out.println(msg);    // Hello
```

② String Pool

- String literals are stored in the **String Pool**.
- If a literal already exists, the same reference is returned.
- Helps in memory optimization.



③ Immutable Strings

- String objects are **immutable** (cannot be changed).
- Any operation that seems to modify the string actually creates a new object.

Example:

```
String s1 = "Hello";
String s2 = s1.concat(" World");
System.out.println(s1);    // Hello
System.out.println(s2);    // Hello World
```

s1 is unchanged.
A new object is created for s2.

④ StringBuilder

- Mutable** class, used to modify strings.
- Not synchronized → not thread-safe.
- More efficient than String for single-threaded environments.

Example:

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming");
sb.insert(5, " is");
sb.delete(0, 1);    // removes 'J'
System.out.println(sb);    // ava is Programming
```

⑤ StringBuffer

- Mutable** class like `StringBuilder`.
- Synchronized → thread-safe.
- Slower than `StringBuilder` but safe for multi-threading.

Example:

```
StringBuffer sb = new StringBuffer("Java");
sb.append(" Programming");
sb.reverse();
System.out.println(sb);    // JavnimargorP avaJ
```

⑥ String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable?	No	Yes	Yes
Thread Safe?	Yes	No	Yes
Performance	Slow	Fast	Slow
Use Case	Less modification	Single-threaded performance critical	Multi-threaded environments

⑦ Common String Methods

Method	Description	Example	Output
<code>length()</code>	Returns length of string	"Java".length()	4
<code>charAt(index)</code>	Returns character at index	"Java".charAt(1)	a
<code>substring(begin, end)</code>	Returns part of string (begin, end)	"Java".substring(1, 3)	av
<code>equals(str)</code>	Compares content	"Java".equals("java")	false
<code>equalsIgnoreCase(str)</code>	Compares ignoring case	"Java".equalsIgnoreCase("java")	true
<code>contains(str)</code>	Checks if sequence exists	"Java Programming".contains("Prog")	true
<code>indexOf(str)</code>	First index of sequence	"Java".indexOf('v')	2
<code>replace(old, new)</code>	Replaces all occurrences	"Java".replace('a', 'o')	Jovo
<code>toUpperCase()</code>	Converts to upper case	"Java".toUpperCase()	JAVA
<code>trim()</code>	Removes leading & trailing spaces	" Java ".trim()	"Java"



Interview Tip

- Strings are immutable and stored in String Pool.
- Use `StringBuilder` for better performance.
- Use `StringBuffer` when thread safety is required.



Best Practices

- Prefer `StringBuilder` over `String` for concatenation in loops.
- Avoid creating unnecessary `String` objects.
- Use meaningful variable names for better readability.

Classes & Objects in Java

Java is an object-oriented programming language.

Everything in Java is an **Object**.

① Class

- A **Class** is a blueprint or template.
- It defines the properties (fields) and behaviors (methods) of objects.
- It does not occupy memory.

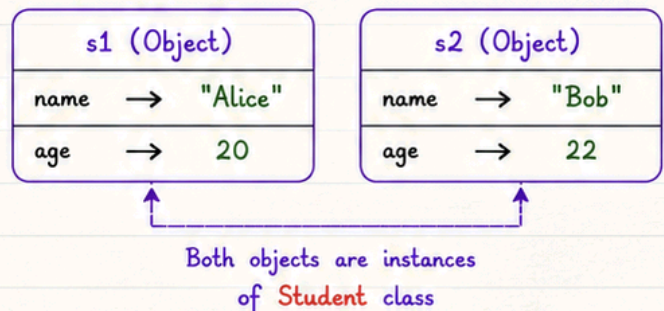
Example: Class Definition

```
class Student {
    String name;           // field
    int age;               // field
    void display() {       // method
        System.out.println(name + " is " + age);
    }
}
```

② Object

- An **Object** is a real-world entity.
- It is an instance of a **Class**.
- It occupies memory in **Heap**.

Example: Objects of Student Class



③ Fields (Variables)

- Fields represent the **state** or **data** of an object.
- Each object has its own copy of fields.

④ Methods

- Methods represent the **behavior** of an object.
- Methods operate on fields and perform actions.
- Methods are defined inside the class.

⑤ Object Creation

- Objects are created using the **new** keyword.

Syntax:

```
ClassName objectName = new ClassName();
```

Example:

```
Student s1 = new Student(); // s1 is an object
Student s2 = new Student(); // s2 is another object
```

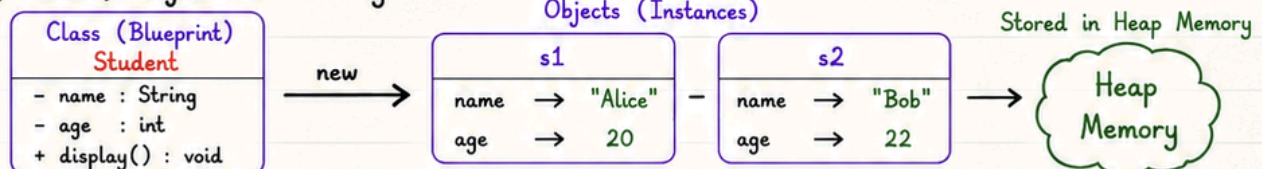
⑥ Accessing Members

- Use dot (.) operator to access fields and methods of an object.

Example:

```
Student s1 = new Student();
s1.name = "Alice";           // access field
s1.age = 20;                 // access field
s1.display();                // call method
```

⑦ Class, Object & Memory



Interview Tip

- Class is a template, Object is an instance.
- Multiple objects can be created from one class.
- Memory for objects is allocated in Heap.



Best Practices

- Design classes around real-world entities.
- Keep fields private and provide public methods (encapsulation).
- Use meaningful names for classes, fields and methods.

Constructors & Keywords in Java

① Constructor

- A **Constructor** is a special method used to initialize objects.
- It has the **same name** as the class.
- It has no return type (not even **void**).
- It is called automatically when an object is created.

Syntax:

```
class ClassName {
    ClassName() {
        // initialization code
    }
}
```



Note

Constructors are used to set initial values for object properties.

② Default Constructor

- A constructor with no parameters.
- If we don't create any constructor, Java provides a default constructor automatically.
- It initializes object with default values.

Example:

```
class Student {
    String name;
    int age;

    // Default constructor (provided by Java)
    Student() { }
}
```

③ Parameterized Constructor

- A constructor that takes parameters.
- Used to initialize object with specific values.

Example:

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

④ Keyword: this

- this** is a reference variable that refers to the current object.
- Used to differentiate instance variables from local variables.

Example:

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name; // refers to instance variable
        this.age = age;   // refers to instance variable
    }
}
```

⑤ Keyword: static

- static** members belong to the **Class**, not to any object.
- Can be accessed using class name.
- Shared among all objects.

Example:

```
class Student {
    static int count = 0; // class variable
    String name;

    Student(String name) {
        this.name = name;
        count++;          // same for all objects
    }
}
```

⑥ Keyword: final

- final** variable: value cannot be changed.
- final** method: cannot be overridden.
- final** class: cannot be inherited.

Example:

```
class Student {
    final String COLLEGE = "ABC Institute"; // constant
    final void show() {
        System.out.println("This method cannot be overridden");
    }
}
```



Interview Tip

- Constructor name must match the class name.
- Constructor has no return type.
- Default constructor is created by JVM if none is defined.

★ Quick Comparison

Keyword	Usage	Belongs to	Can be Changed?
this	Refers to current object	Instance	Yes
static	Class level members	Class	Only once
final	Constant / Restrict changes & inheritance	Depends (variable/method/class)	No



Best Practices

- Use **this** to avoid ambiguity between variables.
- Use **static** for shared data and utility methods.
- Use **final** to make code secure and immutable.

Encapsulation & Inheritance in Java

① Encapsulation

- Encapsulation is the binding of data (fields) and methods that operate on the data into a single unit (class).
- It restricts direct access to some of an object's components which is a way of protecting the data.

★ Benefits

- Data Hiding
- Improved Security
- Better Control on Data
- Increased Flexibility

② Getters & Setters

- Used to access and update **private** fields.
- Getter returns the value.
- Setter validates and sets the value.

Example:

```
class Employee {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

③ Inheritance

- Inheritance is the mechanism where one class acquires (**inherits**) the properties and behaviors of another class.
- It promotes code reusability and establishes **IS-A** relationship.



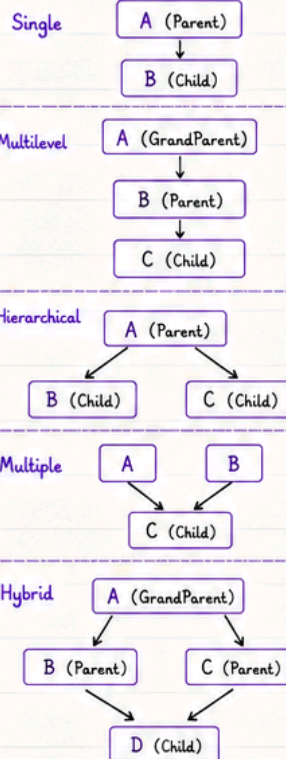
IS-A Relationship

If class B inherits from class A, then B IS-A A.

Example: Dog IS-A Animal

④ Types of Inheritance

- Single Inheritance** - One child inherits from one parent.
- Multilevel Inheritance** - Child inherits from parent, and another child inherits from that child.
- Hierarchical Inheritance** - Multiple children inherit from one parent.
- Multiple Inheritance** - One child inherits from multiple parents. (Not supported in Java with classes, supported with **Interfaces**)
- Hybrid Inheritance** - Combination of two or more types.



⑤ super Keyword

- super** is a reference variable that refers to the immediate parent class object.
- Used to access parent class members (fields, methods, constructors).

Example:

```
class Animal {
    String type = "Animal";
    Animal() { System.out.println("Animal Constructor"); }
}

class Dog extends Animal {
    String type = "Dog";
    Dog() {
        super(); // calls parent constructor
        System.out.println("Dog Constructor");
    }

    void show() {
        System.out.println(super.type); // accesses parent field
    }
}
```

super can be used to:

- ✓ Access parent class fields: **super.fieldName**
- ✓ Access parent class methods: **super.methodName()**
- ✓ Call parent class constructor: **super();**



Interview Tip

- ✓ Encapsulation protects data using **private** + getters/setters.
- ✓ Inheritance models real-world hierarchy using **IS-A**.
- ✓ Choose composition over deep inheritance when possible.



Best Practices

- ✓ Declare fields **private** and provide public access methods.
- ✓ Use meaningful names for classes and methods.
- ✓ Use **super()** in child constructor to initialize parent part.
- ✓ Prefer **Interfaces** over Multiple Inheritance with classes.

Polymorphism & Abstraction in Java

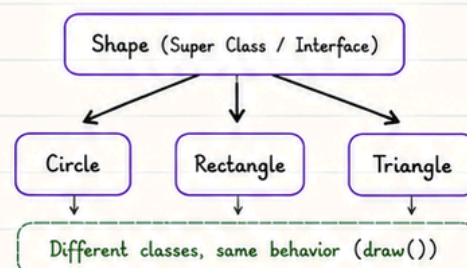
① Polymorphism

- Polymorphism means "many forms".
- It allows objects of different classes to be treated as objects of a common super class.
- Achieved using **Method Overloading** (compile-time) and **Method Overriding** (runtime).



Key Idea

Same method name,
different behavior.
One interface,
multiple implementations.



② Compile-time Polymorphism

- Achieved using **Method Overloading**.
- Same method name, different parameters.
- Resolved by the compiler at compile time.

Example:

```

class Calculator {
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b) { return a + b; }
}
  
```

③ Runtime Polymorphism

- Achieved using **Method Overriding**.
- Same method name and signature in parent and child class.
- Resolved by **JVM** at runtime (dynamic dispatch).

Example:

```

class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
Animal a = new Dog(); // Parent reference
a.sound(); // Bark (runtime polymorphism)
  
```

④ Abstract Class

- Can have both abstract and concrete methods.
- Cannot be instantiated.
- Used to provide a common base and partial implementation.

Example:

```

abstract class Shape {
    abstract double area(); // abstract method
    void display() { // concrete method
        System.out.println("This is a shape");
    }
}
  
```

⑤ Interface

- Contains only abstract methods (before Java 8).
- From Java 8, can have default and static methods.
- Fully abstract = multiple inheritance of type.

Example:

```

interface Drawable {
    void draw(); // abstract method
    default void info() { // default method (Java 8+)
        System.out.println("Drawable interface");
    }
}
  
```

⑥ Method Overriding

- Child class provides specific implementation of parent class method.
- Must have same method name, same parameters, and same return type.
- Access level cannot be more restrictive.

Example:

```

class Parent {
    void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    @Override
    void show() { System.out.println("Child"); }
}
  
```



Interview Tip

- Overloading is about parameters.
- Overriding is about inheritance.
- Prefer interfaces for defining contracts.

★ Quick Comparison

Feature	Compile-time Polymorphism	Runtime Polymorphism
Also Called	Static Polymorphism	Dynamic Polymorphism
Achieved By	Method Overloading	Method Overriding
Binding Time	Compile Time	Runtime
Method Signature	Different	Same
Inheritance	Not Required	Required
Example	add(int, int)	sound() in Parent & Child



Best Practices

- Use abstraction to hide implementation and show behavior.
- Program to interfaces, not to implementations.
- Use @Override annotation always for method overriding.

Exception Handling in Java

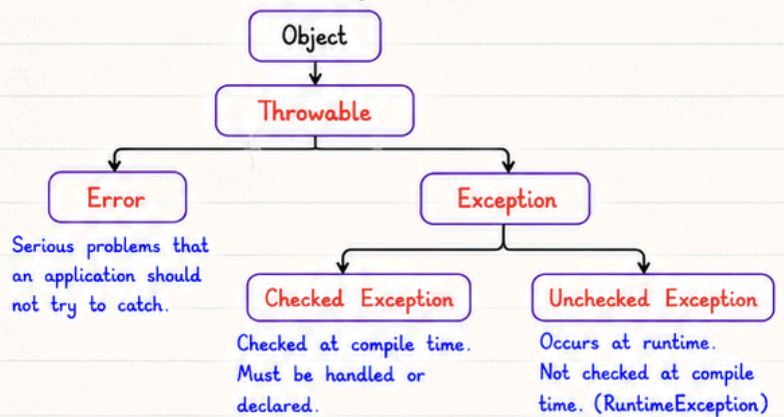
① What is Exception?

An **Exception** is an unwanted or unexpected event that disrupts the normal flow of a program.

★ Benefits

- Improves code reliability
- Avoids abnormal termination
- Separates error handling logic
- Helps in better debugging

② Exception Hierarchy



③ try - catch - finally

- **try** : Contains risky code.
- **catch** : Handles the exception.
- **finally** : Always executes (optional).

```
try {
    int result = 10/0; // risky code
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Finally block always runs");
}
```

④ throw Keyword

- Used to explicitly throw an exception.
- You can throw any Object that is a subclass of **Throwable**.

Example:

```
public void validate(int age) {
    if (age < 18) {
        throw new IllegalArgumentException("Age must be 18 or above");
    }
}
```

⑤ throws Keyword

- Used in method signature to declare exception.
- It shifts the responsibility to the caller.

Example:

```
public void readFile() throws
IOException {
    FileReader fr = new FileReader("file.txt");
}
```

⑥ Custom Exception

- We can create our own exception by extending **Exception** class.

Example:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) {
        super(msg);
    }
}

// Usage
if (age < 0) {
    throw new InvalidAgeException("Age cannot be negative");
}
```

⑦ Common Exceptions

Exception	Description	Type
ArithmeticException	Divide by zero, invalid math operations	Unchecked
NullPointerException	Accessing methods on null object	Unchecked
ArrayIndexOutOfBoundsException	Invalid index in array	Unchecked
NumberFormatException	Invalid conversion of String to number	Unchecked
IOException	Input/Output related exceptions	Checked
ClassNotFoundException	Class not found at runtime	Checked



Interview Tip

- Always prefer specific exceptions over general Exception.
- Use finally to release resources (DB, files, streams).



Best Practices

- Never ignore exceptions.
- Log meaningful error messages.
- Do not catch Exception unless necessary.
- Use custom exceptions to represent business rules.

Packages & Access Modifiers in Java

① Packages

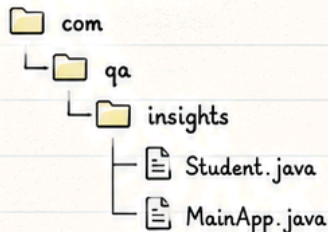
- A **Package** is a namespace that organizes related classes and interfaces.
- Helps in avoiding naming conflicts and improves code maintainability.
- Built-in packages: **java.lang**, **java.util**, **java.io**, etc.

★ Benefits

- ✓ Avoids name conflicts
- ✓ Better organization
- ✓ Easier code maintenance
- ✓ Access control

② Creating & Using Packages

Package Structure



Package Declaration (in Student.java)

```
package com.qa.insights;
```

Importing a Class

```
import com.qa.insights.Student;
```

③ import Statement

- Used to use classes from other packages.

Types:

- Single type import

```
import java.util.ArrayList;
```

- On-demand import

```
import java.util.*;
```

- Static import (Java 5+)

```
import static java.lang.Math.*;
```

💡 Note

Classes from **java.lang** package are imported automatically.

④ Access Modifiers

Modifier	Within Class	Within Package	Subclasses (in other package)	Outside Package	Best For
public	Yes	Yes	Yes	Yes	Maximum accessibility
private	Yes	No	No	No	Data hiding
protected	Yes	Yes	Yes	No	Inheritance
default (no modifier)	Yes	Yes	No	No	Package level access

⑤ Examples

```
// Public - accessible everywhere
public class PublicClass {}
```

```
// Private - accessible only within class
class PrivateClass {
    private int secret = 10;
}
```

```
// Protected - accessible in package
// and by subclasses
```

```
class Parent {
    protected void show() {}
}
```

```
// Default - accessible only in package
class DefaultClass {}
```

⑥ Access in Different Scenarios

Scenario	public	protected	default	private
Same Class	✓	✓	✓	✓
Same Package	✓	✓	✓	✗
Subclass (other package)	✓	✓	✗	✗
Other Package (non-subclass)	✓	✗	✗	✗



Interview Tip

- Always prefer least privilege access.
- Use private for fields and provide getters/setters.
- Default access should be used carefully.

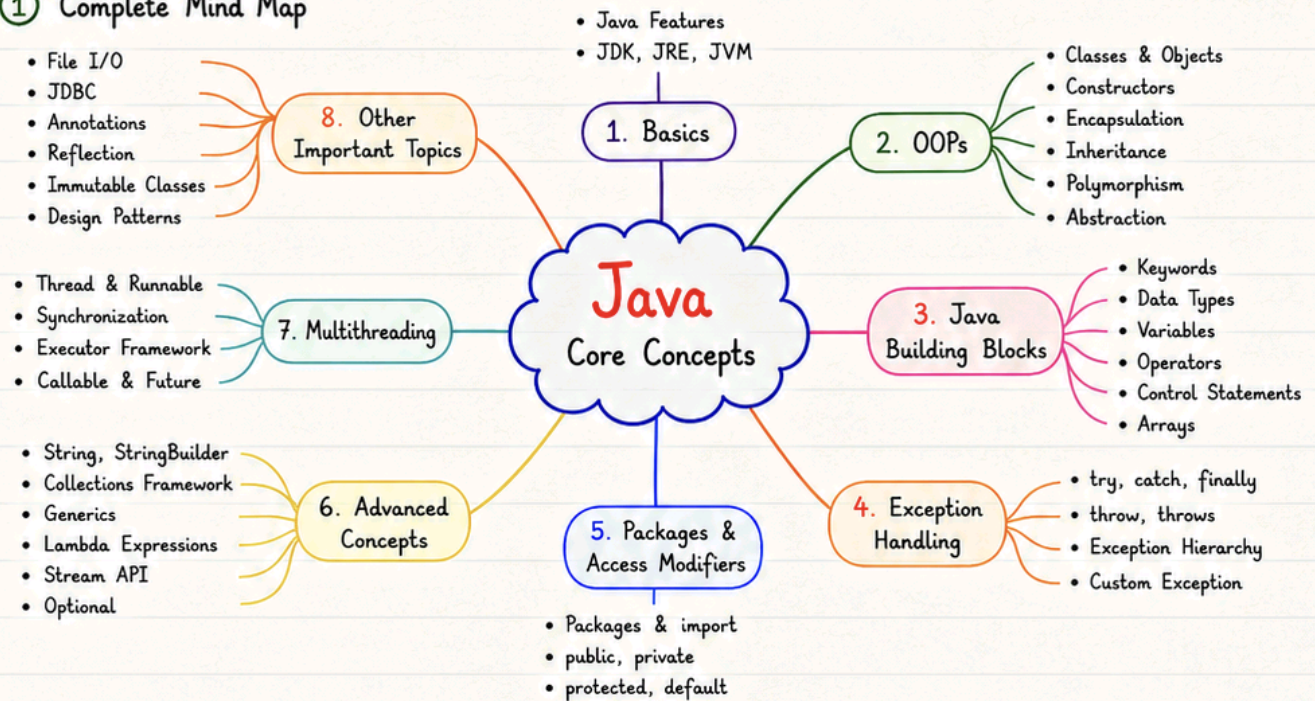


Best Practices

- Organize related classes into meaningful packages.
- Use clear package names (e.g., **com.company.module**).
- Avoid wildcard imports (*), use specific imports.

Java Complete Notes - Quick Revision (Part 1)

① Complete Mind Map



② Top Interview Questions

1. What is Java and how does it work?
 2. What is JVM? Explain its components.
 3. Difference between JDK, JRE and JVM?
 4. What is the difference between == and equals()?
 5. Explain OOPs concepts with examples.
 6. What is polymorphism? Types?
 7. Difference between abstract class and interface?
 8. What is the use of static keyword?
 9. What is exception handling? Explain flow.
 10. Difference between throw and throws?
 11. What are wrappers? Why are they used?
 12. What is String pool in Java?
 13. What is collections framework?
 14. What is multithreading? How to create a thread?
 15. What is synchronization? Why is it needed?
- ...and many more!

③ Best Practices

- ✓ Follow naming conventions.
- ✓ Use meaningful variable and method names.
- ✓ Prefer immutability.
- ✓ Use final wherever possible.
- ✓ Handle exceptions properly, don't ignore them.
- ✓ Follow SOLID principles.
- ✓ Write clean, modular and reusable code.
- ✓ Avoid code duplication.
- ✓ Use appropriate data structures.
- ✓ Comment only where necessary.

④ Common Mistakes

- ✗ Forgetting to close resources (File, DB connections).
- ✗ Catching **Exception** without specific handling.
- ✗ Using **null** without checks (NullPointerException).
- ✗ Not following access modifiers correctly.
- ✗ Using raw types instead of **Generics**.
- ✗ Not overriding **equals()** and **hashCode()** together.
- ✗ Creating objects inside loops unnecessarily.



Quick Revision Tips

- Revise each topic with examples and diagrams.
- Practice coding problems regularly.
- Solve at least 1-2 questions daily.
- Build small projects to strengthen concepts.
- Teach someone - it helps you remember better!



Remember

- ★ Code more. Think logically.
- ★ Understand concepts, don't just memorize.
- ★ Consistency is the key to mastery.
- ★ Make your own short notes.



What's Coming in Part 2?

- ★ Collections Deep Dive
- ★ Streams & Functional Interfaces
- ★ Design Patterns
- ★ JDBC & Database Connectivity
- ★ I/O & NIO
- ★ Interview Q&A (Advanced Level)
- ★ Real-time Project Ideas
- ★ Cheat Sheets & Quick Notes

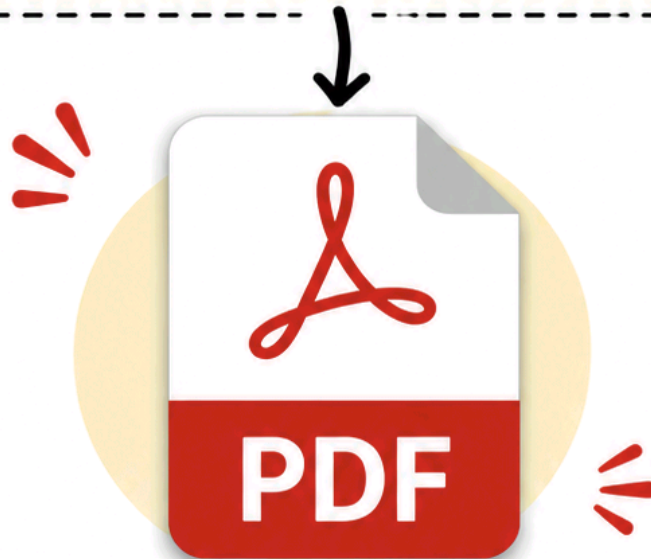
Stay tuned! 🚀



— QUALITY INSIGHTS, BETTER TESTING! —

FOLLOW **& COMMENT** — TO GET —

THE PDF



..... @qa_insights

TESTING INSIGHTS | TOOLS | TUTORIALS | CAREER TIPS | RESOURCES